

Chapitre 1 : Fondamentaux des bases de données et des SGBD

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Base de données 1, fondements, modélisation et SQL · **Fil rouge** : la clinique Santé Plus · **Version** : 2026-10-31

Fiche du chapitre

Objectifs (OS1)	A la fin du chapitre, l'apprenant sait : 1) distinguer une donnée d'une information ; 2) expliquer les limites d'une gestion par fichiers et ce qu'apporte une base de données ; 3) définir une base de données et un SGBD, et citer les rôles et les composants d'un SGBD ; 4) situer les grandes étapes de l'histoire des bases de données et reconnaître les principaux modèles de données (hiérarchique, réseau, relationnel, objet, NoSQL) ; 5) décrire l'architecture à trois niveaux et l'architecture client-serveur ; 6) citer les utilisateurs et les métiers de la donnée ; 7) créer une première table, y insérer des lignes et les lire en SQL.
Prérequis	Savoir utiliser un ordinateur et un navigateur ; notions d'algorithmique
Durée	🟢 Débutant : 8 h · 🔵 Standard : 6 h · 🟣 Expert : 4 h (atelier compris)
Matériel	Un SGBD installé (MySQL ou PostgreSQL) ou un navigateur pour la voie Oracle (FreeSQL)
Livrable	L'atelier 1 : le script `atelier1.sql` et une capture d'écran des résultats

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas. A sauter si le public est à l'aise.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Tout au long du cours, nous construisons la base de données d'une clinique imaginaire de Dakar, **la clinique Santé Plus** : ses patients, ses médecins, ses consultations. Dans ce premier chapitre, on comprend **pourquoi** une telle base est nécessaire, puis on crée sa première table.

Trois façons de travailler

Le cours est écrit pour fonctionner sur **trois SGBD**. Choisissez-en un avec votre enseignant, et gardez-le tout le semestre :

Voie	Ce qu'il faut	Pour qui
MySQL (ou MariaDB, livré avec XAMPP)	Une installation sur votre ordinateur	Le plus répandu dans le développement web
PostgreSQL	Une installation sur votre ordinateur (avec pgAdmin)	Le plus fidèle au standard SQL
Oracle	Un navigateur et un compte gratuit oracle.com : FreeSQL (https://freesql.com), ou la base Oracle de l'école si elle est ouverte	Le SGBD prévu par le syllabus, très présent dans les grandes entreprises

Quand la syntaxe diffère d'un SGBD à l'autre, un encadré vous le signale. Sinon, **le même code fonctionne partout**.

---

1. Donnée et information

Mot	Ce qu'il veut dire	Exemple à la clinique
Donnée	Un fait brut, enregistré, sans interprétation	`Diop`, `1998-04-12`, `38.9`
Information	Une ou plusieurs données **mises en contexte**, qui ont un sens pour quelqu'un	« Awa Diop, née le 12 avril 1998, a 38,9 °C de fièvre ce matin »
Connaissance	Ce qu'on tire de nombreuses informations pour décider	« Les consultations pour paludisme doublent en septembre : prévoyons plus de médecins »

Une base de données stocke des **données** ; ce sont les requêtes, les rapports et les applications qui en font des **informations**.

> 🟢 **Débutant : pourquoi cette distinction ?** La donnée `38.9` toute seule ne sert à rien : est-ce une température, un prix, un poids ? Une base de données bien conçue range chaque donnée **à sa place** (dans la colonne « température » de la bonne consultation), pour qu'on puisse toujours retrouver son sens.

2. Avant les bases de données : les fichiers

2.1 Comment fait une clinique sans base de données

Imaginons la clinique Santé Plus avant l'informatisation de ses données. Chaque service tient **ses propres fichiers** :

- l'accueil : `patients.xlsx` (nom, prénom, téléphone, adresse) ;
- les médecins : un cahier ou un fichier `consultations.docx` par médecin ;
- la caisse : `paiements.xlsx` (nom du patient, montant, date) ;
- la pharmacie : `ordonnances.txt`.

Chaque fichier est indépendant. Chaque logiciel (ou chaque personne) connaît **le format** de son fichier et le lit à sa façon. C'est ce qu'on appelle une **gestion par fichiers**.

2.2 Les sept limites de la gestion par fichiers

Limite	Ce qui se passe à la clinique
1. Redondance	Le nom et le téléphone d'Awa Diop sont recopiés dans `patients.xlsx`, `consultations.docx` et `paiements.xlsx`
2. Incohérence	Awa change de numéro. L'accueil le corrige, la caisse non : quel est le bon numéro ?
3. Accès difficile	« Quels patients de Thiès ont consulté un pédiatre en 2026 ? » : il faut ouvrir trois fichiers et tout recouper à la main
4. Pas de sécurité fine	Le stagiaire de l'accueil peut ouvrir les diagnostics ; impossible de dire « il voit les noms mais pas les diagnostics »
5. Pas de partage simultané	Deux secrétaires ouvrent `patients.xlsx` en même temps ; la dernière qui enregistre écrase le travail de l'autre
6. Pas d'intégrité	Rien n'empêche d'enregistrer un paiement pour un patient qui n'existe pas, ou une date de naissance au 30 février
7. Évolution difficile	Ajouter le groupe sanguin oblige à modifier chaque fichier et chaque programme qui les lit

2.3 Ce qu'apporte une base de données

Une **base de données** rassemble toutes ces données **en un seul endroit**, organisées et reliées, et un logiciel spécialisé, le **SGBD**, se charge de les gérer. Chaque limite trouve sa réponse :

Limite des fichiers	Réponse de la base de données
Redondance	Chaque donnée est stockée **une seule fois** (le patient, une seule fois) ; les autres tables y **font référence**
Incohérence	Une seule copie : une seule correction suffit
Accès difficile	Un langage de requête, **SQL**, répond en une instruction
Sécurité	Des **droits** par utilisateur et par table
Partage	Le SGBD gère les **accès simultanés** (concurrence)
Intégrité	Des **contraintes** vérifiées automatiquement (identifiant unique, date valide, patient existant)
Évolution	On ajoute une colonne sans réécrire les programmes qui ne l'utilisent pas

3. Base de données et SGBD

3.1 Définitions

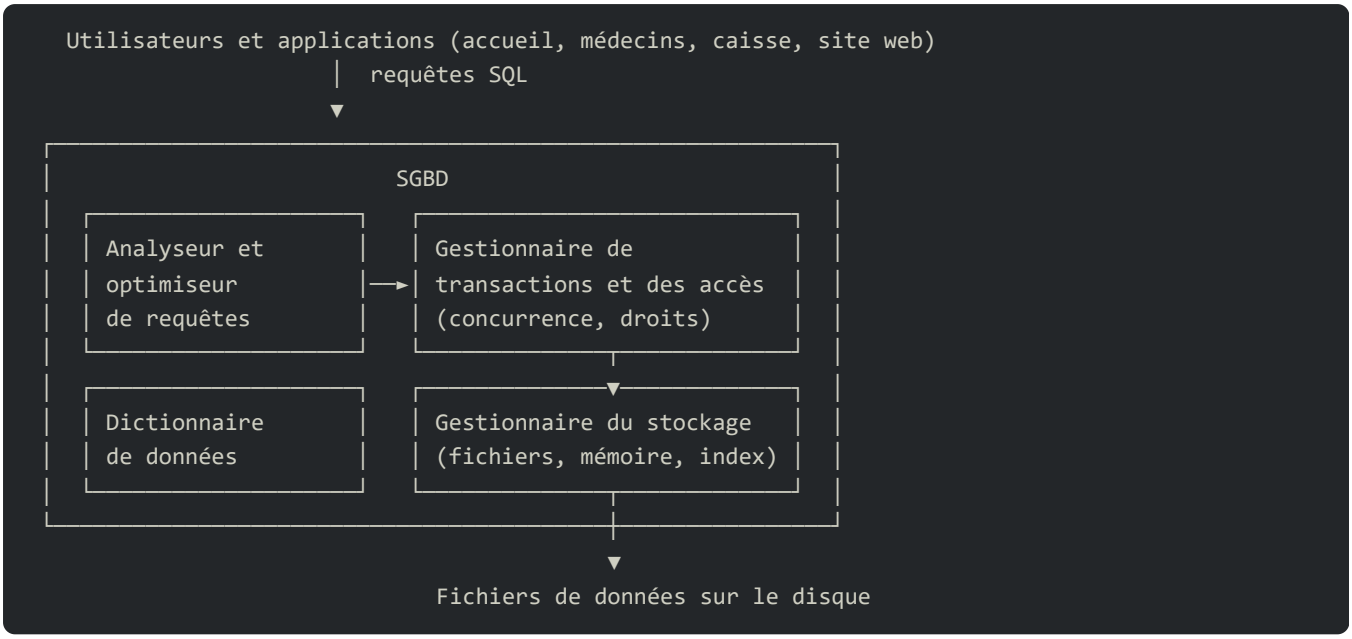
Mot	Définition
Base de données (BD)	Un ensemble **structuré** de données, **reliées entre elles**, stockées de façon durable, et partagées par plusieurs utilisateurs et applications
SGBD (Système de Gestion de Bases de Données)	Le **logiciel** qui crée la base, y range les données, les retrouve, les protège et contrôle qui y accède. Exemples : MySQL, PostgreSQL, Oracle Database, SQL Server, SQLite
SQL (*Structured Query Language*, langage de requêtes structuré)	Le langage normalisé qu'on utilise pour **parler** à un SGBD relationnel : créer des tables, ajouter, modifier, supprimer et rechercher des données

> 🟢 **Débutant : la bibliothèque et le bibliothécaire** La **base de données**, ce sont les livres bien rangés dans les rayons. Le **SGBD**, c'est le bibliothécaire : vous ne fouillez pas vous-même les rayons, vous lui **demandez** un livre (une requête SQL), et il sait où le trouver, vérifie que vous avez le droit de l'emprunter, et note qui l'a pris.

3.2 Les rôles d'un SGBD

Rôle	Ce que fait le SGBD	A la clinique
Stocker	Range les données sur le disque, de façon durable	Les dossiers des patients survivent à une coupure de courant
Retrouver	Exécute les requêtes, le plus vite possible	Retrouver un patient parmi 50000 en une fraction de seconde
Garantir l'intégrité	Vérifie les règles (contraintes) à chaque modification	Refuser une consultation pour un médecin qui n'existe pas
Gérer la concurrence	Permet à plusieurs utilisateurs de travailler en même temps sans se gêner	L'accueil et la caisse travaillent en même temps
Sécuriser	Contrôle qui peut lire ou modifier quoi	La caisse voit les montants, pas les diagnostics
Sauvegarder et restaurer	Permet de revenir à un état sain après une panne ou une erreur	Restaurer la base d'hier après un disque défaillant

3.3 Les composants d'un SGBD



Composant	Son rôle
Analyseur et optimiseur de requêtes	Vérifie que la requête est correcte, puis choisit **la façon la plus rapide** de l'exécuter
Gestionnaire de transactions et des accès	Fait en sorte que plusieurs utilisateurs travaillent en même temps sans abîmer les données, et vérifie les droits
Gestionnaire du stockage	Lit et écrit les données sur le disque, garde en mémoire celles qui servent souvent, gère les index
Dictionnaire de données (ou catalogue)	La « base de la base » : la description de toutes les tables, colonnes, contraintes et utilisateurs

4. Un peu d'histoire, et les modèles de données

Un **modèle de données** est la façon dont un SGBD organise les données et leurs liens. Il en existe plusieurs, apparus au fil de l'histoire.

Époque	Étape	Repère
Années 1960	Fichiers, puis premiers SGBD **hiérarchiques**	IMS d'IBM, conçu pour le programme spatial Apollo
Fin des années 1960	Modèle **réseau**	Normes CODASYL
1970	Le modèle **relationnel**	Article d'Edgar F. Codd (IBM), *A Relational Model of Data for Large Shared Data Banks*
1979	Premier SGBD relationnel commercial utilisant SQL	Oracle (version 2)

Époque	Étape	Repère
1986	SQL devient une **norme**	Norme ANSI, puis ISO
Années 1990	SGBD **objet** et relationnels-objet ; arrivée de MySQL (1995) et de PostgreSQL (1996)	
Années 2000-2010	Bases **NoSQL**, pour les très gros volumes du web	MongoDB, Cassandra, Redis, Neo4j
Aujourd'hui	Bases dans le *cloud*, gérées automatiquement	Oracle Autonomous Database, Amazon RDS…

Voici la même information, « Awa Diop a consulté le Dr Ba », vue par chaque modèle :

Modèle	Organisation	Image
Hiérarchique	Un **arbre** : chaque élément a un seul parent	Médecin Ba → ses consultations → la consultation d'Awa. Si Awa voit deux médecins, elle est recopiée sous chacun
Réseau	Un **graphe** : un élément peut avoir plusieurs parents, reliés par des pointeurs	La consultation d'Awa est reliée à la fois au Dr Ba et à Awa ; pour la trouver, le programme suit les pointeurs
Relationnel	Des **tables** (lignes et colonnes), reliées par des **valeurs communes**	Une table `patient`, une table `medecin`, une table `consultation` qui contient le numéro d'Awa et celui du Dr Ba
Objet	Des **objets** comme en programmation orientée objet (attributs et méthodes)	Un objet `Consultation` qui contient une référence vers l'objet `Patient`
NoSQL	Plusieurs familles : **documents** (JSON), **clé-valeur**, **colonnes**, **graphes**	Un document JSON `{ "patient": "Awa Diop", "consultations": [ { "medecin": "Ba", ... } ] }`

Ce cours porte sur le **modèle relationnel**, de loin le plus utilisé dans les entreprises : MySQL, PostgreSQL, Oracle, SQL Server et SQLite sont tous relationnels.

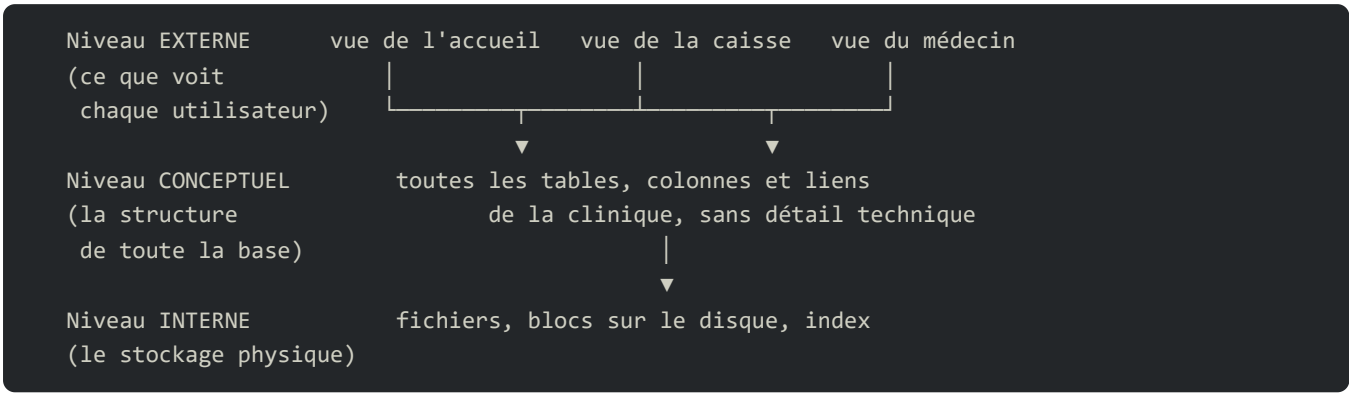
> 🔵 **Standard : pourquoi le relationnel a gagné** Dans les modèles hiérarchique et réseau, le programme doit savoir **comment** les données sont rangées et suivre les liens un par un. Avec le modèle relationnel, on dit seulement **ce qu'on veut** (« les patients de Thiès ») et le SGBD trouve tout seul **comment** l'obtenir. C'est l'**indépendance** entre les données et les programmes (section 5).

> 🟣 **Expert : NoSQL ne remplace pas le relationnel** Les bases NoSQL sont utiles pour des volumes énormes, des données sans structure fixe, ou des réseaux de relations (réseaux sociaux). Mais elles relâchent souvent certaines garanties (cohérence immédiate, contraintes). Dans une clinique, une banque ou une école, où chaque donnée doit être exacte, le relationnel reste la référence. Beaucoup d'entreprises utilisent les deux.

5. L'architecture d'une base de données

5.1 Les trois niveaux (ANSI/SPARC)

En 1975, le comité ANSI/SPARC a proposé de décrire une base de données à **trois niveaux** :



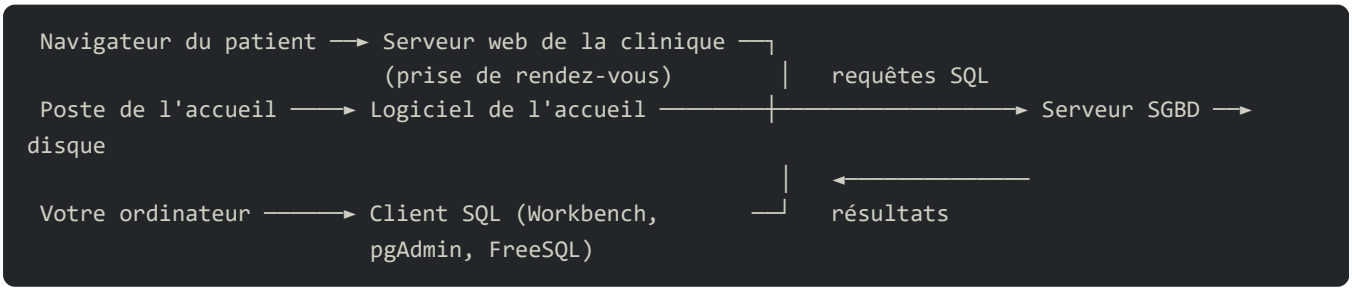
Niveau	Question à laquelle il répond	Qui s'en occupe
Externe	Que voit **cet** utilisateur ? (la caisse voit les montants, pas les diagnostics)	Les développeurs d'applications
Conceptuel (ou logique)	Quelles données existent, et comment sont-elles reliées ?	Le concepteur de la base (chapitres 2 et 3)
Interne (ou physique)	Comment les données sont-elles rangées sur le disque ?	Le SGBD et l'administrateur

Cette séparation apporte deux **indépendances** :

- **indépendance physique** : on peut changer le stockage (nouveau disque, nouvel index) sans toucher aux tables ni aux programmes ;
- **indépendance logique** : on peut ajouter une colonne ou une table sans modifier les vues et les programmes qui ne l'utilisent pas.

5.2 L'architecture client-serveur

Dans la réalité, le SGBD tourne sur un **serveur**, et les applications sont ses **clients** :



Quand vous travaillez sur votre ordinateur avec MySQL ou PostgreSQL, le client et le serveur sont **sur la même machine** (on dit *localhost*). Avec FreeSQL, le serveur Oracle est dans le *cloud* et votre navigateur est le client.

6. Les utilisateurs et les métiers de la donnée

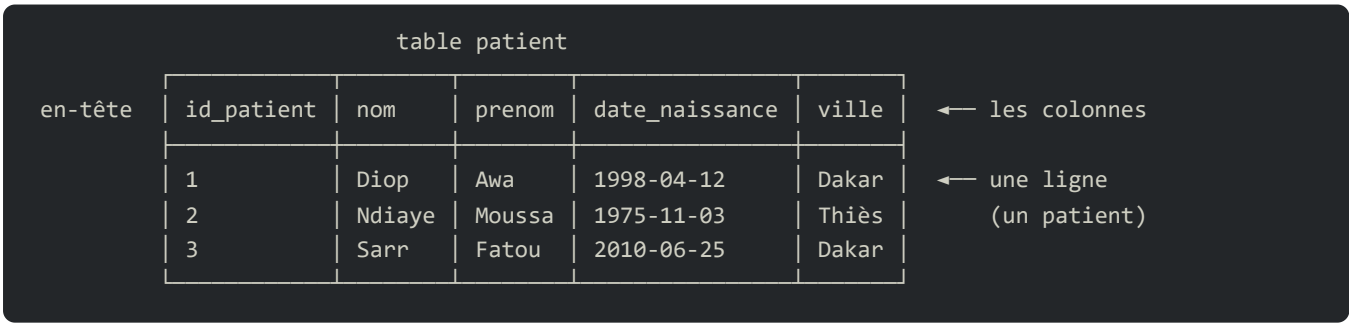
Rôle	Ce qu'il fait	A la clinique
Utilisateur final	Utilise une application, sans écrire de SQL	La secrétaire de l'accueil
Développeur	Écrit les applications qui envoient des requêtes au SGBD	Le développeur du site de prise de rendez-vous
Concepteur de bases de données	Analyse le besoin et dessine le modèle de données	Celui qui décide des tables `patient`, `medecin`, `consultation`
Administrateur de bases de données (DBA)	Installe le SGBD, gère les droits, les sauvegardes, les performances	Il restaure la base après une panne
Analyste de données (*data analyst*)	Interroge les données pour produire des tableaux de bord	« Combien de consultations par service ce mois-ci ? »
Ingénieur de données (*data engineer*)	Construit les chaînes qui collectent et transforment les données	Il alimente chaque nuit l'entrepôt de données du ministère

Ce cours vous donne les bases des trois premiers métiers techniques : **concevoir** (chapitre 2), **créer** (chapitre 3) et **interroger** (chapitre 4).

7. Premier contact avec SQL

7.1 Le vocabulaire d'une table

Dans le modèle relationnel, les données sont rangées dans des **tables** :



Mot	Synonymes	Ce qu'il veut dire
Table	relation	Un ensemble de lignes qui décrivent la même sorte de chose (des patients)
Colonne	attribut, champ	Une information que possède chaque ligne, avec un **type** (texte, nombre, date)
Ligne	enregistrement, tuple	Une chose précise : un patient
Clé primaire	identifiant	La colonne dont la valeur est **unique** pour chaque ligne : `id_patient`. Deux patients peuvent s'appeler Awa Diop ; ils n'auront jamais le même numéro

7.2 Créer une table, la remplir, la lire

Ouvrez votre outil SQL (voir l'atelier 1 pour l'installation), puis tapez et exécutez ce script :

```sql
CREATE TABLE patient (
    id_patient     INT PRIMARY KEY,
    nom            VARCHAR(50) NOT NULL,
    prenom         VARCHAR(50) NOT NULL,
    date_naissance DATE,
    ville          VARCHAR(40)
);

INSERT INTO patient VALUES (1, 'Diop', 'Awa', DATE '1998-04-12', 'Dakar');
INSERT INTO patient VALUES (2, 'Ndiaye', 'Moussa', DATE '1975-11-03', 'Thiès');
INSERT INTO patient VALUES (3, 'Sarr', 'Fatou', DATE '2010-06-25', 'Dakar');

SELECT nom, prenom, ville FROM patient;
SELECT nom, prenom FROM patient WHERE ville = 'Dakar';
```

Résultat (ici avec PostgreSQL ; MySQL affiche les mêmes valeurs, encadrées de `+---+`) :

```
  nom   | prenom | ville
--------+--------+-------
 Diop   | Awa    | Dakar
 Ndiaye | Moussa | Thiès
 Sarr   | Fatou  | Dakar
(3 lignes)

 nom  | prenom
------+--------
 Diop | Awa
 Sarr | Fatou
(2 lignes)
```

Morceau	Ce qu'il veut dire
`CREATE TABLE patient ( ... );`	Crée une table nommée `patient`. Entre parenthèses, une colonne par ligne, séparées par des **virgules**
`id_patient INT PRIMARY KEY`	Une colonne de nombres entiers (`INT`), qui est la **clé primaire** : unique et obligatoire
`VARCHAR(50)`	Du texte de **50 caractères au plus** (*variable character*)
`NOT NULL`	Valeur **obligatoire** : on ne peut pas créer un patient sans nom
`DATE`	Une date (année, mois, jour)
`INSERT INTO patient VALUES (...);`	Ajoute **une ligne**. Les valeurs sont dans l'ordre des colonnes de la table
`'Diop'`	Un texte s'écrit entre **apostrophes** (guillemets simples)

Morceau	Ce qu'il veut dire
`DATE '1998-04-12'`	Une date s'écrit au format **année-mois-jour**, précédée du mot `DATE`
`SELECT nom, prenom, ville FROM patient;`	« Donne-moi les colonnes `nom`, `prenom` et `ville` de la table `patient` ». `SELECT *` donnerait toutes les colonnes
`WHERE ville = 'Dakar'`	Ne garde que les lignes qui respectent la **condition**
`;`	Termine chaque instruction

Ces quatre mots, `CREATE`, `INSERT`, `SELECT`, `WHERE`, sont le cœur de SQL. Les chapitres 3 et 4 les détaillent tous.

> 🟢 **Débutant : SQL se lit presque comme une phrase** `SELECT nom, prenom FROM patient WHERE ville = 'Dakar';` se lit : « sélectionne le nom et le prénom, depuis la table patient, là où la ville vaut Dakar ». Les mots clés (`SELECT`, `FROM`…) peuvent s'écrire en minuscules ; on les écrit en majuscules par habitude, pour les distinguer des noms de tables et de colonnes.

> 🔶 **Oracle** Ce script fonctionne tel quel sur Oracle (FreeSQL ou Autonomous Database). Oracle conseille toutefois `VARCHAR2(50)` plutôt que `VARCHAR(50)`, et range les nombres dans le type `NUMBER` (`INT` y est accepté et devient `NUMBER(38)`). Avec Oracle, il n'y a pas de `CREATE DATABASE` à faire : votre compte a déjà son propre espace (son **schéma**), où créer vos tables.

7.3 Les premières erreurs

Un SGBD est **strict** : la moindre faute est refusée, avec un message. Apprenez à les lire. Partez de la table `patient` et de son patient n°1.

> 🖍️ **Essayez** : `INSERT INTO patient VALUES (4, Gueye, 'Ibou', DATE '2001-01-15', 'Dakar');` (sans apostrophes autour de `Gueye`).
>
> - MySQL : `ERROR 1054 (42S22): Unknown column 'Gueye' in 'field list'`
> - PostgreSQL : `ERREUR: la colonne « gueye » n'existe pas`
>
> Traduction : « colonne inconnue ». Sans apostrophes, le SGBD croit que `Gueye` est un **nom de colonne**. Un texte s'écrit toujours entre apostrophes.

> 🖍️ **Essayez** : `SELECT * FROM patients;` (avec un **s**).
>
> - MySQL : `ERROR 1146 (42S02): Table 'clinique.patients' doesn't exist` (le nom de votre base apparaît avant le point)
> - PostgreSQL : `ERREUR: la relation « patients » n'existe pas`
>
> La table s'appelle `patient` : un nom de table s'écrit exactement.

🖍️ **Essayez** : `INSERT INTO patient VALUES (1, 'Fall', 'Ibou', DATE '2001-01-15', 'Dakar');` (le numéro 1 est déjà pris).

- MySQL : `ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'`
- PostgreSQL : `ERREUR: la valeur d'une clé dupliquée rompt la contrainte unique « patient_pkey »` puis `DÉTAIL : La clé « (id_patient)=(1) » existe déjà.`

La clé primaire **garantit** que deux patients n'ont jamais le même numéro. C'est la limite n°6 des fichiers (l'intégrité), réglée par le SGBD.

🖍️ **Essayez** : `INSERT INTO patient VALUES (5, NULL, 'Ibou', DATE '2001-01-15', 'Dakar');`

- MySQL : `ERROR 1048 (23000): Column 'nom' cannot be null`
- PostgreSQL : `ERREUR: une valeur NULL viole la contrainte NOT NULL de la colonne « nom » dans la relation « patient »`

`NULL` veut dire « aucune valeur ». La colonne `nom` est `NOT NULL` : refusé.

🖍️ **Essayez** : `INSERT INTO patient VALUES (6, 'Fall', 'Ibou', DATE '2001-02-30', 'Dakar');`

- MySQL : `ERROR 1525 (HY000): Incorrect DATE value: '2001-02-30'`
- PostgreSQL : `ERREUR: valeur du champ date/time en dehors des limites : « 2001-02-30 »`

Le 30 février n'existe pas : le type `DATE` vérifie les dates. Un fichier Excel, lui, l'aurait accepté comme du texte.

🖍️ **Essayez** : `SELECT nom, prenom, FROM patient;` (une virgule de trop).

- MySQL : `ERROR 1064 (42000): You have an error in your SQL syntax; ... near 'FROM patient' at line 1`
- PostgreSQL : `ERREUR: erreur de syntaxe sur ou près de « FROM »`

Une **erreur de syntaxe** : la phrase SQL est mal construite. Le message indique **où** le SGBD s'est arrêté : regardez juste avant.

🔵 **Standard : apostrophes ou guillemets ?** Écrivez `SELECT nom FROM patient WHERE ville = "Dakar";` avec des **guillemets doubles**. MySQL l'accepte (il renvoie Awa Diop et Fatou Sarr), mais PostgreSQL répond `ERREUR: la colonne « Dakar » n'existe pas`, et Oracle refuse aussi. Dans la norme SQL, les guillemets doubles entourent des **noms** (de colonnes, de tables), et les apostrophes des **textes**. Prenez dès maintenant la bonne habitude : **apostrophes pour les textes**, et votre code fonctionnera sur tous les SGBD.

> 🔵 **Standard : majuscules et minuscules dans les textes** `SELECT COUNT(*) FROM medecin WHERE specialite = 'pédiatrie';` (avec un **p** minuscule) trouve les pédiatres sur MySQL (qui ne distingue pas, par défaut, majuscules et minuscules), mais **aucun** sur PostgreSQL et Oracle, pour qui `'pédiatrie'` et `'Pédiatrie'` sont deux textes différents. Écrivez les valeurs **exactement** comme elles ont été enregistrées.

Exercices

Pour les exercices 4 et 5, utilisez votre SGBD (dans une base, ou un schéma, à vous).

- ★ **Exercice 1 : donnée ou information ?** Pour chacun de ces éléments, dites s'il s'agit d'une donnée, d'une information ou d'une connaissance, et justifiez en une phrase : a) `221` ; b) « le vol Dakar-Ziguinchor du 3 novembre part à 7 h 30 » ; c) « les ventes de bissap augmentent pendant le Ramadan : commandons plus d'hibiscus » ; d) `2026-11-03` ; e) « l'étudiant n°1342 a 14,5 de moyenne au premier semestre ». (Section 1.)
- ★ **Exercice 2 : les limites des fichiers.** Une auto-école de Saint-Louis gère ses élèves dans un cahier, ses leçons de conduite dans un fichier Excel par moniteur, et ses paiements dans un autre fichier. Donnez **cinq** problèmes concrets que cette organisation va poser, en nommant à chaque fois la limite (redondance, incohérence…), puis dites comment une base de données règle chacun. (Section 2.)
- ★ **Exercice 3 : rôles et niveaux.** Pour chaque situation, dites quel **rôle du SGBD** (section 3.2) ou quel **niveau** de l'architecture (section 5.1) est concerné : a) le comptable de la clinique ne doit pas voir les diagnostics ; b) l'administrateur ajoute un disque plus rapide et rien ne change pour les utilisateurs ; c) deux caissiers encaissent au même moment ; d) le serveur tombe en panne, on récupère les données de la veille ; e) on refuse un rendez-vous à une date inexistante.
- ★★ **Exercice 4 : la table des livres.** Créez une table `livre` pour une bibliothèque de quartier à Rufisque : numéro (clé primaire), titre (obligatoire, 100 caractères au plus), auteur (obligatoire), année de parution, nombre d'exemplaires. Insérez cinq livres d'auteurs africains de votre choix (au moins deux du même auteur, et un sans année de parution). Écrivez les requêtes qui affichent : a) tous les livres ; b) le titre et l'auteur seulement ; c) les livres d'un auteur donné ; d) les livres parus après 2000 ; e) les livres dont l'année est inconnue (cherchez « SQL IS NULL »). (Section 7.)
- ★★★ **Exercice 5 : provoquer et expliquer.** Sur votre table `livre`, provoquez **volontairement** cinq erreurs différentes (au moins une d'intégrité, une de syntaxe, une de type). Pour chacune, recopiez la requête, le message exact de **votre** SGBD, sa traduction, et la correction. Si vous en avez la possibilité, exécutez les mêmes requêtes sur un deuxième SGBD (par exemple FreeSQL) et comparez les messages. (Section 7.3.)

Atelier 1 : installer son SGBD et créer la table des médecins

Vous installez votre SGBD (ou ouvrez FreeSQL), vous créez la base de la clinique et sa table des médecins, vous la remplissez et vous l'interrogez. L'énoncé de l'atelier vous est fourni séparément.

A retenir

- Une **donnée** est un fait brut ; une **information** est une donnée mise en contexte.
- La gestion par **fichiers** souffre de sept limites : redondance, incohérence, accès difficile, pas de sécurité fine, pas de partage simultané, pas d'intégrité, évolution difficile.
- Une **base de données** est un ensemble structuré de données reliées et partagées ; le **SGBD** est le logiciel qui la gère : il stocke, retrouve, garantit l'intégrité, gère la concurrence, sécurise, sauvegarde.
- Modèles de données : hiérarchique, réseau, **relationnel** (Codd, 1970 : des tables reliées par des valeurs), objet, NoSQL.
- Architecture à **trois niveaux** (externe, conceptuel, interne) : indépendance physique et logique. En pratique, architecture **client-serveur**.
- Une **table** a des **colonnes** (typées) et des **lignes** ; la **clé primaire** identifie chaque ligne de façon unique.
- SQL : `CREATE TABLE`, `INSERT INTO ... VALUES`, `SELECT ... FROM ... WHERE`. Textes entre **apostrophes**, dates en `DATE 'AAAA-MM-JJ'`, `;` à la fin.

Erreurs fréquentes

Message (MySQL / PostgreSQL)	Ce que ça veut dire	Solution
`Unknown column 'X' in 'field list'` / `la colonne « x » n'existe pas`	Un texte écrit sans apostrophes, ou une colonne mal orthographiée	Mettre le texte entre apostrophes ; vérifier le nom de la colonne
`Table '...' doesn't exist` / `la relation « x » n'existe pas`	Nom de table mal écrit, ou table pas encore créée (ou créée dans une autre base)	Vérifier l'orthographe ; exécuter d'abord le `CREATE TABLE` ; vérifier la base utilisée
`Duplicate entry '1' for key 'PRIMARY'` / `la valeur d'une clé dupliquée rompt la contrainte unique`	Deux lignes avec la même clé primaire	Choisir un autre numéro
`Column 'x' cannot be null` / `une valeur NULL viole la contrainte NOT NULL`	Valeur manquante pour une colonne obligatoire	Donner une valeur
`Incorrect DATE value` / `valeur du champ date/time en dehors des limites`	Date impossible, ou mal écrite	Format `DATE 'AAAA-MM-JJ'`, date existante
`You have an error in your SQL syntax ... near '...'` / `erreur de syntaxe sur ou près de « ... »`	Phrase SQL mal construite (virgule, parenthèse, mot clé)	Regarder juste **avant** l'endroit indiqué

Message (MySQL / PostgreSQL)	Ce que ça veut dire	Solution
`Column count doesn't match value count` (MySQL)	Pas autant de valeurs que de colonnes dans le `INSERT`	Donner une valeur par colonne, dans l'ordre (PostgreSQL complète silencieusement les dernières colonnes par `NULL`)
`WHERE ville = "Dakar"` marche sur MySQL, pas ailleurs	Guillemets doubles au lieu d'apostrophes	Apostrophes pour les textes

Pour aller plus loin

- Documentation MySQL, *Tutorial* : https://dev.mysql.com/doc/refman/8.4/en/tutorial.html
- Documentation PostgreSQL en français, *Tutoriel* : https://docs.postgresql.fr/16/tutorial.html
- Oracle FreeSQL (Oracle dans le navigateur) : https://freesql.com
- Oracle Academy, *Applied Database Systems*, section 1 (support du syllabus, fourni par l'enseignant si l'école y a accès).
- Livre de référence : R. Elmasri et S. Navathe, *Fundamentals of Database Systems*, Pearson, chapitres 1 et 2.